

Module 1

Kotlin Deep Dive

Type System & Language Core

- Declaration-site variance & Star projection
- Platform Types & Intersection Types
- Sealed Classes/Interfaces & Exhaustive when
- Contracts
- Operator Overloading & Convention Methods
- Destructuring Internals
- Infix Functions & Extension Function Resolution

Functions & Functional Style

- inline / noinline / crossinline — Bytecode Impact
- Scope Functions — When Each Actually Fits
- Local Functions & Closures & tailrec

Classes, Initialization & Lifecycle

- Initialization Order & Leaking this
- lateinit Internals
- Delegated Properties
- companion object vs Top-Level & const

Equality, Immutability & State

- `==` vs `===` & `equals`/`hashCode` Contracts
- Data Class copy Pitfalls
- `val` $\neq$ `Immutable`
- The Cast Attack: `List` $\rightarrow$ `MutableList`
- Backing Properties Beyond Flow

Collections & Performance

- Sequences vs Collections
- Primitive Arrays
- ArrayDeque & Persistent Collections
- Lesser-Known Operators

Coroutines

- Structured Concurrency & Job Hierarchy
- Dispatchers & Context Switching
- Cooperative Cancellation
- Exception Handling
- Flow: Cold/Hot, Operators & Backpressure
- Channels vs Flow & select
- Mutex & Semaphore vs synchronized
- Coroutine Internals: CPS & State Machines

Concurrency Beyond Coroutines

- JMM, @Volatile & Atomics
- Thread-Blocking Traps in Coroutines

Error Modeling

- `Result<T>` & the `runCatching` Trap
- Checked Exceptions Interop & `@Throws`
- Typed Errors: Either-Style

Compiler, Interop & Metaprogramming

- KAPT vs KSP
- Java Interop Pitfalls
- Show Kotlin Bytecode — Recurring Demo Device
- kotlin-reflect: Cost & R8
- Annotations: Retention & Targets
- Compiler Plugins: How Compose & Serialization Work

Modern Kotlin & Ecosystem

- K2 Compiler & Smarter Smart Casts
- New Language Features (2.2+)
- Rich Errors / Union Types Roadmap
- Multiplatform Awareness
- Tooling & Build